EXP9

September 02, 2026

[ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
```

[ ]:
```
df = pd.read_csv("adult.csv")

# Data exploration
print("First Five Records\n")
print(df.head())

print("last Five Records\n")
print(df.tail())

print("\nDataset Information\n")
print(df.info())

print("\nDataset Description\n")
print(df.describe())

print("\nDataset Shape")
print(df.shape)
```

```
First Five Records

   age  workclass  fnlwgt     education  educational-num      marital-status  \
```

```
0   25    Private  226802          11th                7       Never-married
1   38    Private   89814       HS-grad                9  Married-civ-spouse
2   28  Local-gov  336951    Assoc-acdm               12  Married-civ-spouse
3   44    Private  160323  Some-college               10  Married-civ-spouse
4   18          ?  103497  Some-college               10       Never-married

          occupation relationship   race  gender  capital-gain  capital-loss  \
0  Machine-op-inspct    Own-child  Black    Male             0             0
1    Farming-fishing      Husband  White    Male             0             0
2    Protective-serv      Husband  White    Male             0             0
3  Machine-op-inspct      Husband  Black    Male          7688             0
4                  ?    Own-child  White  Female             0             0

   hours-per-week native-country income
0              40  United-States  <=50K
1              50  United-States  <=50K
2              40  United-States   >50K
3              40  United-States   >50K
4              30  United-States  <=50K
last Five Records

       age     workclass  fnlwgt   education  educational-num  \
48837   27       Private  257302  Assoc-acdm               12
48838   40       Private  154374     HS-grad                9
48839   58       Private  151910     HS-grad                9
48840   22       Private  201490     HS-grad                9
48841   52  Self-emp-inc  287927     HS-grad                9

           marital-status         occupation relationship   race  gender  \
48837  Married-civ-spouse       Tech-support         Wife  White  Female
48838  Married-civ-spouse  Machine-op-inspct      Husband  White    Male
48839             Widowed       Adm-clerical    Unmarried  White  Female
48840       Never-married       Adm-clerical    Own-child  White    Male
48841  Married-civ-spouse    Exec-managerial         Wife  White  Female

       capital-gain  capital-loss  hours-per-week native-country income
48837             0             0              38  United-States  <=50K
48838             0             0              40  United-States   >50K
48839             0             0              40  United-States  <=50K
48840             0             0              20  United-States  <=50K
48841         15024             0              40  United-States   >50K

Dataset Information

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48842 entries, 0 to 48841
Data columns (total 15 columns):
 #   Column           Non-Null Count  Dtype
```

```
---  ------           --------------  -----
 0   age              48842 non-null  int64
 1   workclass        48842 non-null  object
 2   fnlwgt           48842 non-null  int64
 3   education        48842 non-null  object
 4   educational-num  48842 non-null  int64
 5   marital-status   48842 non-null  object
 6   occupation       48842 non-null  object
 7   relationship     48842 non-null  object
 8   race             48842 non-null  object
 9   gender           48842 non-null  object
 10  capital-gain     48842 non-null  int64
 11  capital-loss     48842 non-null  int64
 12  hours-per-week   48842 non-null  int64
 13  native-country   48842 non-null  object
 14  income           48842 non-null  object
dtypes: int64(6), object(9)
memory usage: 5.6+ MB
None

Dataset Description

                age        fnlwgt  educational-num  capital-gain  \
count  48842.000000  4.884200e+04     48842.000000  48842.000000
mean      38.643585  1.896641e+05        10.078089   1079.067626
std       13.710510  1.056040e+05         2.570973   7452.019058
min       17.000000  1.228500e+04         1.000000      0.000000
25%       28.000000  1.175505e+05         9.000000      0.000000
50%       37.000000  1.781445e+05        10.000000      0.000000
75%       48.000000  2.376420e+05        12.000000      0.000000
max       90.000000  1.490400e+06        16.000000  99999.000000

       capital-loss  hours-per-week
count  48842.000000    48842.000000
mean      87.502314       40.422382
std      403.004552       12.391444
min        0.000000        1.000000
25%        0.000000       40.000000
50%        0.000000       40.000000
75%        0.000000       45.000000
max     4356.000000       99.000000

Dataset Shape
(48842, 15)
```

[ ]:
```
df.columns = df.columns.str.strip()
```

```
df.replace("?", np.nan, inplace=True)

for column in df.select_dtypes(include="object").columns:
    df[column] = df[column].str.strip()

print("\nMissing Values")
print(df.isnull().sum())

df.dropna(inplace=True)

df.drop_duplicates(inplace=True)

df.reset_index(drop=True, inplace=True)

print("\nDataset Shape After Removing Missing Values:")
print(df.shape)
```

```
Missing Values
age                  0
workclass         2799
fnlwgt               0
education            0
educational-num      0
marital-status       0
occupation        2809
relationship         0
race                 0
gender               0
capital-gain         0
capital-loss         0
hours-per-week       0
native-country     857
income               0
dtype: int64

Dataset Shape After Removing Missing Values:
(45175, 15)
```

[ ]:
```
encoder = LabelEncoder()

for column in df.columns:
    if df[column].dtype == "object":
        df[column] = encoder.fit_transform(df[column])

print("\nColumn Names:")
print(df.columns)
```

```
X = df.drop("income", axis=1)
y = df["income"]
```

```
Column Names:
Index(['age', 'workclass', 'fnlwgt', 'education', 'educational-num',
       'marital-status', 'occupation', 'relationship', 'race', 'gender',
       'capital-gain', 'capital-loss', 'hours-per-week', 'native-country',
       'income'],
      dtype='object')
```

[ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (36140, 14)
Testing Data Shape: (9035, 14)
```

[ ]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

[ ]:
```
logistic_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)
logistic_model.fit(X_train_scaled, y_train)

y_pred_lr = logistic_model.predict(X_test_scaled)
```

[ ]:
```
decision_tree_model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=5,
    random_state=42
)
decision_tree_model.fit(X_train, y_train)

y_pred_dt = decision_tree_model.predict(X_test)
```

[ ]:
```
def evaluate_model(y_test, y_pred, model_name):

    accuracy = accuracy_score(y_test, y_pred)

    precision = precision_score(
        y_test,
        y_pred,
        average="weighted",
        zero_division=0
    )

    recall = recall_score(
        y_test,
        y_pred,
        average="weighted",
        zero_division=0
    )

    f1 = f1_score(
        y_test,
        y_pred,
        average="weighted",
        zero_division=0
    )

    print(model_name)
    print("Accuracy :", accuracy)
    print("Precision:", precision)
    print("Recall   :", recall)
    print("F1 Score :", f1)

    print("\nClassification Report:")
    print(classification_report(
        y_test,
        y_pred,
        zero_division=0
    ))

    return accuracy, precision, recall, f1

lr_results = evaluate_model(
    y_test,
    y_pred_lr,
    "LOGISTIC REGRESSION"
)

dt_results = evaluate_model(
```

```
    y_test,
    y_pred_dt,
    "DECISION TREE"
)

comparison = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Decision Tree"
    ],
    "Accuracy": [
        lr_results[0],
        dt_results[0]
    ],
    "Precision": [
        lr_results[1],
        dt_results[1]
    ],
    "Recall": [
        lr_results[2],
        dt_results[2]
    ],
    "F1 Score": [
        lr_results[3],
        dt_results[3]
    ]
})

print("MODEL COMPARISON")
print(comparison)
```

```
LOGISTIC REGRESSION
Accuracy : 0.8148312119535142
Precision: 0.8024155622955101
Recall   : 0.8148312119535142
F1 Score : 0.7995801900619552

Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.94      0.88      6795
           1       0.70      0.45      0.54      2240

    accuracy                           0.81      9035
   macro avg       0.77      0.69      0.71      9035
weighted avg       0.80      0.81      0.80      9035

DECISION TREE
```

```
Accuracy : 0.8426120641947981
Precision: 0.8366133303082204
Recall   : 0.8426120641947981
F1 Score : 0.8296281113232561

Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.96      0.90      6795
           1       0.79      0.50      0.61      2240

    accuracy                           0.84      9035
   macro avg       0.82      0.73      0.76      9035
weighted avg       0.84      0.84      0.83      9035

MODEL COMPARISON
                 Model  Accuracy  Precision    Recall  F1 Score
0  Logistic Regression  0.814831   0.802416  0.814831  0.799580
1        Decision Tree  0.842612   0.836613  0.842612  0.829628
```

[ ]:
```python
plt.figure(figsize=(20, 10))

plot_tree(
    decision_tree_model,
    feature_names=X.columns,
    class_names=["<=50K", ">50K"],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title("Decision Tree for Adult Income Prediction")
plt.show()
```

Decision Tree for Adult Income Prediction

